

Service Provider Platform

Backend API Documentation

Complete Developer Guide

Generated on: 28/1/2026

Service Provider Platform - Backend API Documentation

Table of Contents

1. [Overview](#overview)
2. [Base URL & Authentication](#base-url--authentication)
3. [Environment Configuration](#environment-configuration)
4. [Database Models](#database-models)
5. [API Endpoints](#api-endpoints)
6. [Error Handling](#error-handling)
7. [Response Format](#response-format)

Overview

This is a comprehensive backend API for a Service Provider Platform built with Node.js, Express.js, and MongoDB. The platform connects customers with service providers and includes features like property management, booking system, payment integration, and admin dashboard.

Tech Stack

- **Runtime**: Node.js
- **Framework**: Express.js
- **Database**: MongoDB with Mongoose ODM
- **Authentication**: JWT (JSON Web Tokens)
- **File Upload**: Cloudinary
- **Payment Gateway**: Razorpay
- **Email Service**: Nodemailer

Key Features

- User & Vendor Authentication
- Property Management
- Booking System

- Payment Integration (Razorpay)
- Category Management
- Blog Management
- Rating & Review System
- Admin Dashboard
- Audit Logs
- Customer Support

Base URL & Authentication

Base URL

`http://localhost:8000/api/v1`

Authentication

The API uses JWT (JSON Web Token) for authentication. Include the token in the Authorization header:
 Authorization: Bearer <your-jwt-token>

Environment Configuration

Required Environment Variables

`JWT_SECRET=your_jwt_secret`
`MONGODB_URL=your_mongodb_connection_string`
`PORT=8000`

Email Configuration

`MAIL_HOST=smtp.gmail.com`
`MAIL_USER=your_email@gmail.com`
`MAIL_PASS=your_app_password`
`ADMIN_EMAIL=admin@example.com`

Cloudinary Configuration

`CLOUD_NAME=your_cloudinary_name`
`API_KEY=your_cloudinary_key`
`API_SECRET=your_cloudinary_secret`
`FOLDER_NAME=your_folder_name`

Razorpay Configuration

`RAZORPAY_KEY=your_razorpay_key`
`RAZORPAY_SECRET=your_razorpay_secret`

Database Models

1. Auth Model (Users & Admins)

```
{
  name: String,
```

```

email: String,
phone: String,
password: String,
role: ["user", "admin"], // default: "user"
token: String,
type: String, // default: "active"

// Permission flags
isVendor: Boolean,
isBlog: Boolean,
isUser: Boolean,
isSupport: Boolean,
isJob: Boolean,
isAds: Boolean,
isBooking: Boolean,
isEmpManage: Boolean,
isCategoryManage: Boolean,
isManageService: Boolean,

timestamps: true
}

```

2. Vendor Model

```

{
// Basic Info
name: String,
email: String,
password: String,
phone: String,
company: String,
address: String,
description: String,

// Service Details
typeOfService: String,
category: ObjectId (ref: Category),
subCategory: String,
yearOfEstablishment: String,
serviceLocation: String,
alternatePhone: String,
whatsappNumber: String,
businessType: ["Proprietorship", "Partnership", "LLP", "Private Limited", "Other"],
gstNumber: String,
tradeLicense: String,
numberOfStaff: Number,
referralCode: String,
referralName: String,
workingDaysTimings: String,

// System Fields
role: "vendor",

```

```

isAdmin: Boolean,
status: String,
percentage: String,
adhar: String,
pan: String,
token: String,
updateProfileRequest: ["pending", "requested", "approved"],

// Working Hours
workingHours: {
  monday: { start: String, end: String, available: Boolean },
  tuesday: { start: String, end: String, available: Boolean },
  // ... for all days
},

// Bank Details
bankDetail: {
  accountNumber: String,
  IFSC: String,
  accountHolderName: String,
  branch: String
},

// Experience
experience: {
  fields: [String],
  totalYears: Number
},

// Documents
profilePhoto: String,
document1: String,
document2: String,
document3: String,
document4: String,
document5: String,

timestamps: true
}

```

3. Property Model

```

{
  title: String (required),
  price: String (required),
  location: String (required),
  type: String (required),
  category: String (required),
  description: String,
  status: ["active", "inactive"], // default: "active"
  images: [{
    public_id: String,

```

```
url: String
}},
vendor: ObjectId (ref: Vendor, required),
review: [ObjectId] (ref: RatingAndReview),
timestamps: true
}
```

API Endpoints

Authentication Routes (`/api/v1/auth`)

POST `/login`

Description: User/Admin login

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "jwt_token_here",
  "user": {
    "id": "user_id",
    "name": "User Name",
    "email": "user@example.com",
    "role": "user"
  }
}
```

POST `/register`

Description: User/Admin registration

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "1234567890",
  "password": "password123"
}
```

GET `/getAll`

Description: Get all users (Admin only)

Response: Array of user objects

PUT `/update/:id`

Description: Update user permissions

Parameters: `id` - User ID

****Request Body****: Permission flags to update

DELETE `/delete/:id`

****Description****: Delete user account

****Parameters****: `id` - User ID

GET `/my-profile/:id`

****Description****: Get user profile and inquiries

****Parameters****: `id` - User ID

PUT `/change-type/:id`

****Description****: Change user type/status

****Parameters****: `id` - User ID

PUT `/change-password/:id`

****Description****: Change user password

****Parameters****: `id` - User ID

Vendor Routes (`/api/v1/vendor`)

POST `/login`

****Description****: Vendor login

****Request Body****:

```
{  
  "email": "vendor@example.com",  
  "password": "password123"  
}
```

POST `/register`

****Description****: Vendor registration

****Request Body****:

```
{  
  "name": "Vendor Name",  
  "email": "vendor@example.com",  
  "password": "password123",  
  "phone": "1234567890",  
  "company": "Company Name",  
  "typeOfService": "Service Type",  
  "address": "Complete Address"  
}
```

GET `/getAll`

****Description****: Get all vendors

****Response****: Array of vendor objects

GET `/get/:id`

****Description****: Get vendor by ID

****Parameters****: `id` - Vendor ID

PUT `/update/:id`

****Description**:** Update vendor status

****Parameters**:** `id` - Vendor ID

PUT `/update-profile/:id`

****Description**:** Update vendor profile

****Parameters**:** `id` - Vendor ID

PUT `/update-percentage/:id`

****Description**:** Update vendor commission percentage

****Parameters**:** `id` - Vendor ID

PUT `/working-hours/:id`

****Description**:** Update vendor working hours

****Parameters**:** `id` - Vendor ID

****Request Body**:**

```
{
  "workingHours": {
    "monday": { "start": "09:00", "end": "18:00", "available": true },
    "tuesday": { "start": "09:00", "end": "18:00", "available": true }
  }
}
```

POST `/request-update/:id`

****Description**:** Request profile update approval

****Parameters**:** `id` - Vendor ID

DELETE `/delete/:id`

****Description**:** Delete vendor account

****Parameters**:** `id` - Vendor ID

Property Routes (`/api/v1/property`)

POST `/create`

****Description**:** Create new property

****Request Body**:**

```
{
  "title": "Property Title",
  "price": "50000",
  "location": "City, State",
  "type": "Residential",
  "category": "Apartment",
  "description": "Property description",
  "vendor": "vendor_id"
}
```

POST `/get-vendor-property`

****Description**:** Get properties by vendor

****Request Body**:**

```
{
```

```
"vendorId": "vendor_id"
}
```

PUT `/update/:id`

****Description****: Update property details

****Parameters****: `id` - Property ID

PUT `/update-status/:id`

****Description****: Update property status

****Parameters****: `id` - Property ID

GET `/getAll`

****Description****: Get all properties

****Response****: Array of property objects

GET `/get/:id`

****Description****: Get property by ID

****Parameters****: `id` - Property ID

DELETE `/delete/:id`

****Description****: Delete property

****Parameters****: `id` - Property ID

Booking Routes (`/api/v1/booking`)

POST `/create`

****Description****: Create new booking

****Request Body****:

```
{
  "vendorId": "vendor_id",
  "customerId": "customer_id",
  "serviceDate": "2024-01-15",
  "serviceTime": "10:00 AM",
  "description": "Service description",
  "amount": 1000
}
```

GET `/getAll`

****Description****: Get all bookings (Admin)

****Response****: Array of booking objects

GET `/get/:vendorId`

****Description****: Get bookings by vendor

****Parameters****: `vendorId` - Vendor ID

PUT `/update/:bookingId`

****Description****: Update booking status

****Parameters****: `bookingId` - Booking ID

****Request Body****:

```
{
```



```
"status": "confirmed" // or "cancelled", "completed"
}
```

Payment Routes (`/api/v1/razorpay`)

POST `/capturePayment`

Description: Create Razorpay order

Request Body:

```
{
  "amount": 1000,
  "currency": "INR",
  "receipt": "receipt_id"
}
```

Response:

```
{
  "success": true,
  "orderId": "order_id",
  "amount": 1000,
  "currency": "INR"
}
```

POST `/verifyPayment`

Description: Verify payment signature

Request Body:

```
{
  "razorpay_order_id": "order_id",
  "razorpay_payment_id": "payment_id",
  "razorpay_signature": "signature"
}
```

Category Routes (`/api/v1/category`)

POST `/create`

Description: Create new category (Admin)

Request Body:

```
{
  "name": "Category Name",
  "description": "Category description",
  "price": 500
}
```

GET `/getAll`

Description: Get all categories

Response: Array of category objects

PUT `/update/:id`

Description: Update category (Admin)

****Parameters**:** `id` - Category ID

DELETE `/delete/:id`

****Description**:** Delete category (Admin)

****Parameters**:** `id` - Category ID

POST `/purchase`

****Description**:** Purchase category by vendor

****Request Body**:**

```
{  
  "vendorId": "vendor_id",  
  "categoryId": "category_id",  
  "paymentMethod": "online" // or "cash"  
}
```

GET `/purchased/:vendorId`

****Description**:** Get purchased categories by vendor

****Parameters**:** `vendorId` - Vendor ID

GET `/purchasers/:categoryId`

****Description**:** Get purchasers for a category (Admin)

****Parameters**:** `categoryId` - Category ID

GET `/pending`

****Description**:** Get all pending purchases (Admin)

GET `/pending/:vendorId`

****Description**:** Get vendor's pending purchases

****Parameters**:** `vendorId` - Vendor ID

PUT `/approve/:purchaseId`

****Description**:** Approve purchase (Admin)

****Parameters**:** `purchaseId` - Purchase ID

PUT `/reject/:purchaseId`

****Description**:** Reject purchase (Admin)

****Parameters**:** `purchaseId` - Purchase ID

Additional Routes

Blog Routes (`/api/v1/blog`)

- CRUD operations for blog management
- Public blog listing and detailed view

Contact Routes (`/api/v1/contact`)

- Contact form submissions
- Admin contact management

Customer Support Routes (`/api/v1/customer-support`)

- Support ticket creation and management
- Admin support dashboard

Job Routes (`/api/v1/job`)

- Job posting and management
- Job application handling

Career Routes (`/api/v1/career`)

- Career opportunity management
- Application processing

Rating & Review Routes (`/api/v1/rating`)

- Service rating and review system
- Review moderation

Ads Routes (`/api/v1/ads`)

- Advertisement management
- Ad placement and tracking

Dashboard Routes (`/api/v1/dashboard`)

- Admin dashboard statistics
- Analytics and reporting

Image Routes (`/api/v1/image`)

- Image upload and management
- Cloudinary integration

Audit Logs Routes (`/api/v1/audit`)

- System activity logging
- Admin audit trail

Error Handling

Standard Error Response Format

```
{
  "success": false,
  "message": "Error description",
  "error": "Detailed error information"
}
```

Common HTTP Status Codes

- `200` - Success
- `201` - Created
- `400` - Bad Request
- `401` - Unauthorized
- `403` - Forbidden
- `404` - Not Found
- `500` - Internal Server Error

Response Format

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": {
    // Response data
  }
}
```

Error Response

```
{
  "success": false,
  "message": "Error message",
  "error": "Detailed error information"
}
```

Security Features

1. **JWT Authentication**: Secure token-based authentication
2. **Password Hashing**: bcryptjs for password encryption
3. **CORS Configuration**: Cross-origin resource sharing setup
4. **Input Validation**: Request data validation
5. **File Upload Security**: Secure file handling with Cloudinary
6. **Environment Variables**: Sensitive data protection

File Upload

The API supports file uploads through Cloudinary integration:

- **Supported formats**: Images (JPG, PNG, GIF, etc.)
- **Upload endpoint**: Integrated with relevant routes
- **Storage**: Cloudinary cloud storage
- **Security**: File type validation and size limits

Payment Integration

Razorpay Integration

- **Test Mode**: Currently configured for testing
- **Supported Methods**: Cards, UPI, Net Banking, Wallets
- **Currency**: INR (Indian Rupees)
- **Security**: Signature verification for payment confirmation

Database Relationships

1. **Vendor !' Category**: Many-to-One relationship
2. **Property !' Vendor**: Many-to-One relationship
3. **Booking !' Vendor**: Many-to-One relationship
4. **Booking !' Customer**: Many-to-One relationship
5. **Property !' Reviews**: One-to-Many relationship

Deployment Notes

Production Checklist

1. Update environment variables for production
2. Enable Razorpay live mode
3. Configure production MongoDB cluster
4. Set up SSL certificates
5. Configure production email settings
6. Update CORS origins for production domains

Server Requirements

- Node.js 14+
- MongoDB 4.4+
- Minimum 1GB RAM
- SSL certificate for HTTPS

This documentation covers the complete backend API for the Service Provider Platform. For any additional information or support, please contact the development team.